

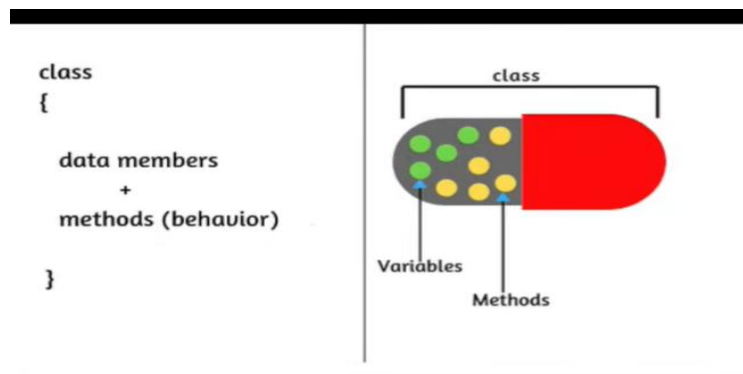
What is Encapsulation and Abstraction?

Encapsulation, abstraction & data hiding is closely related to each other. When we talk about any C++ program, it consists of two fundamental elements:

- **Program statements** – Part of the program (functions) that performs actions.
- **Program data** – Data in the program which is manipulated using functions.

2-1 Encapsulation

Encapsulation binds the data & functions together which keeps both safe from outside interference. Data encapsulation led to data hiding.



Example:

Here is an example of a bank deposit using encapsulation in C++. The `balance` data member is kept `private` to prevent direct modification, while a `public` `deposit()` method provides a controlled interface to interact with the balance, ensuring data integrity.

```
#include <iostream>
using namespace std;
class BankAccount {
private:
    double balance;    // Private data member (data hiding)

public:
    // initialize the account with an opening balance
    BankAccount(double initialBalance) {
        if (initialBalance > 0) {
            balance = initialBalance;
        }
        else {
            balance = 0;
            cout << "Initial balance cannot be negative. Setting to 0." << endl;
        }
    }
};
```

```

    }
}

// Public method to deposit money (controlled access)
void deposit(double amount) {
    if (amount > 0) {
        balance += amount;    // Internal modification is allowed within the class
        cout << "Deposited: $" << amount << ". New balance: $" << balance << endl;
    }
    else {
        cout << "Invalid deposit amount. Amount must be positive." << endl;
    }
}

// Public method (getter) to retrieve the balance
double getBalance() const {
    return balance;
};

```

Const تعني ان الدالة لا تغير أي شيء داخل الكائن فقط تقرأ البيانات

```

int main() {
    // Create a BankAccount object with an initial balance of $1000
    BankAccount myAccount(1000.0);

    // Display initial balance
    cout << "Initial balance: $" << myAccount.getBalance() << endl;

    // Deposit money into the account using the public method
    myAccount.deposit(500.0);

    // Attempt an invalid deposit
    myAccount.deposit(-50.0);

    // Display final balance
    cout << "Final balance: $" << myAccount.getBalance() << endl;

    // The following line would cause a compilation error, demonstrating data hiding:
    // myAccount.balance = 10000; // Error: 'balance' is private

    return 0;
}

```

لاحظ طريقة اسناد القيم لدالة البناء ضمنية (implicit call)

Output:
 Initial balance: \$1000
 Deposited: \$500. New balance: \$1500
 Invalid deposit amount. Amount must be positive.
 Final balance: \$1500

إذا رفعنا تجميد الابعاز سوف يظهر خطأ في مرحلة compilation بسبب محاولة الوصول ال بيانات خاصة.

Explanation for benefits of Encapsulation in BankAccount example

- **Data Hiding:** The `balance` variable is declared as `private`. This means it cannot be accessed or modified directly from outside the `BankAccount` class (e.g., in the `main` function).
- **Bundling Data and Functions:** The data (`balance`) and the function that operates on it (`deposit()`, `getBalance()`) are bound together within a single unit, the `BankAccount` class.
- **Controlled Access:** The `deposit()` method is declared as `public`, providing a specific and controlled way to modify the balance. This allows for validation (e.g., ensuring the deposited amount is positive) before the internal state is changed, which helps maintain data integrity and security.

2-2 Abstraction

Abstraction in C++ involves **hiding the complex implementation details** and showing only the necessary features or an interface to the user. It can be achieved in two main ways: by using **classes with access specifiers** (data abstraction) or by using **abstract classes and pure virtual functions** (control abstraction).



Example(1):

In this example we display Abstraction using Classes and Access Specifiers.

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    int rollNumber; // Hidden data/implementation detail
    string name;    // Hidden data

public:
    // Public function to set values in a controlled manner
    void setDetails(string n, int r) {
        name = n;
        rollNumber = r;
    }

    // Public function to display information (interface)
    void showDetails() {
        cout << "Name: " << name << endl;
        cout << "Roll Number: " << rollNumber << endl;
    }
};

int main() {
    Student s1;
    s1.setDetails("Amit", 101); // User interacts with public method
    s1.showDetails();           // User sees the result, not the internal variables
    // s1.rollNumber = 102;     // Error: rollNumber is private and inaccessible
    return 0;
}
```

Explanation: The `rollNumber` and `name` are private, so they cannot be accessed directly from `main()`. The public methods `setDetails()` and `showDetails()` provide a controlled interface to interact with this data, effectively abstracting the underlying data storage.

Example (2):

Create a **Lamp** class with private variables, and public methods to perform turn on and turn off and change color tasks. Ensure encapsulation and abstraction concepts founded in your program.

```
#include <iostream>
#include <string>
using namespace std;

class Lamp {
private:
    bool isOn;           // state of lamp is hiding
    string color;        // lamp coclor is hiding also

public:
    // constructor assinment is in the body of the dunction
    Lamp() {
        isOn=false;
        color="White";
    } // default color

    void turnOn() {
        isOn = true;
        cout << "Lamp is ON ??, Color: " << color << endl;
    }

    void turnOff() {
        isOn = false;
        cout << "Lamp is OFF ??" << endl;
    }

    void setColor(string newColor) {
        color = newColor;
        cout << "Lamp color changed to: " << color << endl;
    }

    void showColor() {
        cout << "Current lamp color: " << color << endl;
    }
};

int main() {
    Lamp lamp;

    lamp.turnOn();           // turn on the lamp with default
color
    lamp.setColor("Red");    // change the color
    lamp.showColor();        // display the current color
    lamp.turnOff();          // turn of the lamp

    return 0;
}
```

Output:
Lamp is ON ??, Color: White
Lamp color changed to: Red
Current lamp color: Red
Lamp is OFF ??

H.W:

1- create a class **calculator** with a private variable result to store the current result of the operation (addition, multiplication, subtraction, division) the show the result.

2- Develop the **Lamp** program to store how many times the lamp has been turned on.